

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

Members:

YADAV Anshuman Krishna (exercise 4, 5 & 6)

MAHALINGAM Nithees (exercise 1, 2 & 3)

SARAVANAN Arun Prasath (Absent)

Github Link:

https://github.com/anshuman-krishna/advanced-algorithms-and-programming/tree/LAB_01_Revision#

Exercise 1:

Algorithm ReverseInteger(n)

Input: Non-negative integer n

Output: Integer formed by reversing digits of n

Set reversed $\leftarrow$ 0

While n > 0 do

 digit $\leftarrow$ n mod 10

 reversed $\leftarrow$ (reversed $\times$ 10) + digit

 n $\leftarrow$ n div 10

End While

Return reversed

Complexity Analysis Questions:

1. How many arithmetic operations are performed for a d-digit number?

Answer:

Suppose the number has **n** digits.

Each loop iteration performs:

- 1 modulo (%)
- 1 multiplication (*)
- 1 addition (+)
- 1 division (//)

So the total operations are : **4n**

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

2. What is the time complexity (number of expensive operations) in terms of the input value n ?

Answer:

Each digit is processed once.

If the input digit has n digits, then:

$T(n) = O(n)$

So the time complexity is : **$O(n)$**

Code Related Queries:

Reflection:

This algorithm is efficient because:

- It processes each digit exactly once.
- No extra data structures are used.
- Memory usage is constant.

It is optimal for digit reversal using arithmetic operations.

Test Cases:

Input	Output	Reason
315	513	Normal case
400	4	Trailing Zeros Removed
0	0	Edge Case
7	7	Sindle Digit

Exercise 2:

Algorithm IsBalanced(s)

Input: String s containing $()$, $[\]$, $\{ \}$

Output: True if balanced, False otherwise

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

```
Create empty stack S
For each character ch in s do
  If ch is '(' or '[' or '{' then
    Push ch onto stack S
  Else // ch is closing symbol
    If stack S is empty then
      Return False
    top <- Pop from stack S
    If ch is ')' and top ≠ '(' then
      Return False
    If ch is ']' and top ≠ '[' then
      Return False
    If ch is '}' and top ≠ '{' then
      Return False
If stack S is empty then
  Return True
Else
  Return False
```

Complexity Analysis Questions:

1. How many comparisons are needed for an n-character string?

Answer:

For each character:

- We check whether it is an opening bracket.
- If it is a closing bracket, we:
 - Check if the stack is empty.
 - Check if it matches the correct opening bracket on top of the stack.

All of these checks take a constant amount of time for each character (at most a few comparisons).

So, for a string of n characters, we do this same small amount of work **n** times.

That means the total number of operations grows proportionally with **n**.

Therefore, the total time complexity is : **O(n)**

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

2. What happens if we extend to include additional symbol pairs or other different characters?

Answer:

If we add more symbol pairs, nothing changes structurally.

Only change is:

- Adding more matching conditions.

The time complexity remains the same i.e. $O(n)$.

Code Related Queries:

Reflection:

Stack ensures correct nesting (LIFO behavior).

Detects:

- Missing opening symbols
- Extra closing symbols
- Improper nesting

Efficient because it scans string only once.

Test Cases:

Input	Output	Reason
"{}"	True	Proper nesting
"()]"	False	Incorrect nesting
"	True	Empty string
"(((	False	Unmatched Openings
"]"	False	Unmatched closing

Exercise 3:

Algorithm MergeIntervals(intervals)

Input: List of intervals [start, end]

Output: List of merged non-overlapping intervals

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

```
If intervals is empty
  Return empty list
Sort intervals based on start value
Create empty list result
Set current <- intervals[0]
For each interval next from intervals[1] to end do
  If next.start ≤ current.end then
    current.end <- max(current.end, next.end)
  Else
    Add current to result
    current <- next
Add current to result
Return result
```

Complexity Analysis Questions:

1. How many comparison operations are needed for n intervals?

Answer:

There are 2 parts:

a) Sorting:

Sorting takes : $n \log n$

b) Merging Loop:

We iterate once through intervals : $n-1$, so n

The total comparisons : $n \log n + n = n \log n$

2. What is the time complexity of your solution?

Answer:

- Sorting : $O(n \log n)$
- Merging pass : $O(n)$

Overall time complexity : $O(n \log n)$

Code Related Queries:

Reflection:

- Sorting is necessary to ensure correct merging.

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

- After sorting, only one linear scan is required.
- Algorithm is optimal because merging without sorting could lead to $O(n^2)$.

This balances both time and memory efficiency.

Test Cases:

Input	Output	Reason
[[1,3],[2,6]]	[[1,6]]	Overlap
[[1,4],[4,5]]	[[1,5]]	Touching Intervals
[[1,4],[0,4]]	[[0,4]]	Contained Interval
[[1,2],[3,4]]	[[1,4]]	No Overlap
[]	[]	Empty List
[[5,7]]	[[5,7]]	Single Interval

Exercise 4:

```
FUNCTION EvaluatePolynomial(coeffs, x) -> Real
result <- 0

FOR i <- length(coeffs) - 1 DOWNT0 0 DO
result <- result * x + coeffs[i]
END FOR

RETURN result
```

Complexity Analysis Questions:

1. How many multiplications are needed for a degree-n polynomial?

Answer:
n multiplications

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

2. Compare with the naive approach: what is the improvement?

Answer:

Naive approach: Calculating x^k takes k multiplications. Summing that up for all terms gives roughly $n^2/2$ multiplications, hence: $O(n^2)$

Horner's approach: Uses n multiplications, hence, $O(n)$

Improvement: Reduces the complexity by a lot for large polynomials (from Quadratic to Linear)

Code Related Queries:

Complexity

time: $O(n)$

space: $O(1)$

Reflection

Horner's approach reduces repeated power computations and improves performance from quadratic to linear.

Edge cases tested

- empty list
- single coefficient
- zero polynomial
- negative coefficients
- floating/large x

Exercise 5:

Approach A: Using Temporary Array (Linear Space)

```
PROCEDURE RotateTemp(Vect, k)
n <- length(Vect)
```

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

```
k <- k mod n
temp <- empty list

FOR i <- n - k TO n - 1 DO
temp.Add(Vect[i])
END FOR

FOR i <- 0 TO n - k - 1 DO
temp.Add(Vect[i])
END FOR

Vect <- temp
```

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$

Approach B: Rotate One by One (Related to $n*k$ time)

```
PROCEDURE RotateOneByOne(Vect, k)
n <- length(Vect)
k <- k mod n

FOR i <- 1 TO k DO
last <- Vect[n - 1]

FOR j <- n - 1 DOWNT0 1 DO
Vect[j] <- Vect[j - 1]
END FOR

Vect[0] <- last
END FOR
```

- Time Complexity: $O(n*k)$
- Space Complexity: $O(1)$

Approach C: Reverse Segments Method (Linear Time, Constant Space)

```
PROCEDURE RotateReverse(Vect, k)
n <- length(Vect)
k <- k mod n

Reverse(Vect, 0, n - 1)
```

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

Reverse(Vect, 0, k - 1)

Reverse(Vect, k, n - 1)

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

Complexity Analysis Questions:

1. What is the optimal time complexity achievable?

Answer:

The optimal achievable complexity is **$O(n)$** , which is Linear.

2. How does k affect the performance?

Answer:

Since we: $k \bmod n$, a big k wouldn't affect a lot, the performance depends on the size of the array (n) and not k.

3. When would each approach be preferable?

Answer:

- **Temp Array:** Good when memory is cheap and we want very readable code.
- **Rotate One-by-One:** Generally bad. Is only acceptable if k is very small, otherwise it's too slow ($O(n \times k)$).
- **Reverse Segments:** The best choice. It's fast, ($O(n)$) and uses constant memory ($O(1)$).

Code Related Queries:

Complexity

temporary: time $O(n)$, space $O(n)$

one by one: time $O(nk)$, space $O(1)$

reverse: time $O(n)$, space $O(1)$

Reflection

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

The reverse method is optimal,
One-by-one is inefficient for large k ,
The temporary method is simple but uses extra memory.

Edge cases tested

- empty array
- single element
- $k = 0$
- $k > n$
- small arrays

Exercise 6:

Approach A: Using two passes with frequency dictionary

```
FUNCTION FirstUnique(s) -> Integer
freq <- empty dictionary
```

```
FOR EACH ch IN s DO
  IF ch IN freq THEN
    freq[ch] <- freq[ch] + 1
  ELSE
    freq[ch] <- 1
  END IF
END FOR
```

```
FOR i <- 0 TO length(s) - 1 DO
  IF freq[s[i]] = 1 THEN
    RETURN i
  END IF
END FOR
```

```
RETURN -1
```

Approach B: Using ordered dictionary or linked hashed map

```
FUNCTION FirstUniqueOrdered(s) -> Integer
map <- empty dictionary
```

```
FOR i <- 0 TO length(s) - 1 DO
  ch <- s[i]
```

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

```
IF ch IN map THEN
map[ch] <- map[ch] + 1
ELSE
map[ch] <- 1
END IF
END FOR

FOR i <- 0 TO length(s) - 1 DO
IF map[s[i]] = 1 THEN
RETURN i
END IF
END FOR

RETURN -1
```

Complexity Analysis Questions:

1. What is the space complexity of your solution?

Answer:

$O(1)$ (Constant Space)

Even though we are creating a dictionary (freq), the size of that dictionary doesn't depend on how long the string n is. It depends on the number of unique characters in the alphabet.

Whether the input is 10 letters or 10 million, the dictionary will never hold more than 26 keys (assuming lowercase English). Since that limit is fixed, the space is constant.

2. How many passes through the string are needed?

Answer:

Two Passes

- Pass 1: The first for loop where we build the freq map.
- Pass 2: The second for loop where we check if $\text{freq}[s[i]] == 1$.

II.2415 Advanced Algorithms and Programming

Lab 1: Revision of Algorithms' Fundamentals: Team T24

So, total operations are roughly $2n$, which keeps it linear.

3. What data structure is most efficient for this problem?

Answer:

A Hash Table

The core difficulty of this problem is knowing the count of any character instantly. A Hash Table allows for $O(1)$ (constant time) lookups. This means checking if `freq[s[i]] == 1` happens instantly, regardless of how much data is in the map.

If we used a list or array to store counts (without direct indexing like ASCII codes), we would have to search through it, making the algorithm much slower ($O(n^2)$).

Code Related Queries:

Complexity

time: $O(n)$

space: $O(\sigma)$

[In the worst case, σ can be equal to n , so space may become $O(n)$.

If the alphabet size is fixed (e.g., 26 lowercase letters), then space is $O(1)$.]

Reflection

The hash table allows constant lookup,

The two-pass method is reliable and simple,

An ordered behavior ensures correct first occurrence detection.

Edge cases tested

- empty string
- single character
- all repeated
- first character unique
- last character unique

the end.